

Colecciones en Java

Array · ArrayList · HashSet · HashMap

Dominio: Sistema de gestión de una biblioteca universitaria

¿Cómo usar esta guía?

Lee cada bloque completo antes de abrir IntelliJ. Cada paso tiene una pregunta socrática — respóndela mentalmente, luego lee la respuesta guiada. Cuando la guía diga «**Ahora codifica**», cambia a IntelliJ y construye lo que se describe. No hay código en este documento: el razonamiento es tuyo, la implementación también.

Objetivo: al terminar deberás poder justificar — con argumento técnico — por qué usaste cada colección, no solo que el código compile.

LO QUE VAS A CONSTRUIR

Paso	Qué construyes	Colección involucrada
1	Interfaz Catalogable + clase abstracta Material	—
2	Subclases Libro y Revista	—
3	Catálogo de materiales disponibles	HashMap
4	Lista de espera por material	ArrayList
5	Registro de usuarios con membresía activa	HashSet
6	SistemaBiblioteca: integrar todo y ejecutar	Las tres juntas

El problema

La Biblioteca Central de la Universidad del Código necesita un sistema digital básico. Hoy todo se hace en papel: el catálogo de libros y revistas, la lista de personas esperando un ejemplar prestado, y el registro de usuarios con membresía activa.

Tu tarea es construir ese sistema en Java. No vas a resolver todo el dominio — solo el núcleo: los materiales que tiene la biblioteca, las listas de espera, y los miembros registrados. El sistema debe poder responder tres preguntas clave:

- *¿Qué materiales tiene la biblioteca y cuáles están disponibles?*
- *¿Quién está esperando un ejemplar específico — y en qué orden llegaron?*
- *¿Este usuario tiene membresía activa en este momento?*

Cada una de esas preguntas tiene una estructura de datos ideal. Este taller te guía para descubrirla — no para memorizar la respuesta.

1

La base: Catalogable y Material

Interfaz + clase abstracta · Sin colecciones aún

Antes de pensar en colecciones, necesitas definir qué es un material de biblioteca y qué tienen en común todos los materiales — sin importar si son libros o revistas. Aquí entra la programación orientada a objetos.

■ ¿Por qué una interfaz antes de la clase?

Porque la biblioteca necesita un contrato mínimo: todo lo que entre al sistema debe poder identificarse. No importa si es un libro físico, una revista digital o un futuro audiovisual — todos tienen que responder a `getCodigo()`. La interfaz **Catalogable** es ese contrato. Define el 'qué', no el 'cómo'.

Crea la interfaz Catalogable con el método: `String getCodigo()`

■ ¿Qué tienen en común todos los materiales de la biblioteca?

Todo material tiene: un **código único** (para identificarlo en el catálogo), un **título**, un **año de publicación** y un **estado de disponibilidad** (si está prestado o no). Esos atributos no dependen del tipo de material. Por eso van en la clase abstracta **Material**, que implementa **Catalogable**.

■ ¿Por qué Material es abstracta y no concreta?

Porque no puede existir un 'material genérico' — siempre es un libro o una revista específica. La abstracción impide instanciar algo que no tiene sentido sin concretar. Además, necesitamos un método abstracto `getTipo()` que cada subclase implemente de forma distinta: 'Libro' o 'Revista'.

Crea la clase abstracta Material que implemente Catalogable. Atributos: `codigo`, `titulo`, `anio (int)`, `disponible (boolean)`. Incluye constructor, getters, `getCodigo()` implementado y `getTipo()` abstracto. Agrega un método concreto `mostrarInfo()` que imprima: `[tipo] titulo (anio) — disponible/prestado`.

■ **Antes de continuar — verifica esto en tu código:**

`mostrarInfo()` llama a `getTipo()` — y `getTipo()` es abstracto. ¿Cómo funciona? En runtime, el objeto siempre es una subclase concreta (Libro o Revista). Java busca el `getTipo()` de esa subclase específica. Eso es polimorfismo: el método base delega la decisión a quien lo hereda.

Ahora que tienes la base, necesitas los tipos concretos. Cada uno hereda todo de Material y agrega solo lo que le es propio.

■ ¿Qué tiene un Libro que no tiene una Revista?

Un libro tiene un **autor** y un **ISBN**. Una revista tiene un **número de edición** y una **temática** (tecnología, ciencias, artes...). Todo lo demás — código, título, año, disponible — ya vive en Material.

■ ¿Qué hace `super()` en el constructor de la subclase?

Delega la inicialización de los atributos del padre a Material. Sin `super()`, los atributos heredados (código, título, año, disponible) no se inicializarían. Java lo exige como primera instrucción del constructor hijo.

Crea Libro con atributos: autor (String), isbn (String). `getTipo()` retorna "Libro". Crea Revista con atributos: edicion (int), tematica (String). `getTipo()` retorna "Revista".

■ ¿Por qué `@Override` importa?

No es obligatorio para compilar, pero es una promesa al compilador: 'este método existe en el padre y lo estoy reemplazando'. Si cometes un error en el nombre del método (`getTiopo` en lugar de `getTipo`), `@Override` lo detecta en compilación — sin él, crearías un método nuevo sin darte cuenta y el polimorfismo no funcionaría.

VERIFICA ANTES DE CONTINUAR

- Libro y Revista tienen su propio constructor que llama `super()` correctamente.
- `getTipo()` está anotado con `@Override` en ambas subclases.
- Puedes instanciar un Libro y llamar `mostrarInfo()` — debe imprimir [Libro] y el título.
- Puedes instanciar una Revista y ver [Revista] en el output.

3

El catálogo de materiales

HashMap — Primera colección

La biblioteca necesita saber qué materiales tiene y poder encontrar uno de forma inmediata cuando alguien lo pide. No tiene sentido recorrer toda la colección para encontrar el libro con código 'BK-2024-001'. Necesitas acceso directo.

REGLAS CLAVE DEL HashMap

Regla	Qué significa en la práctica
Claves únicas	No puede haber dos materiales con el mismo código. Si registras el mismo código dos veces, el segundo reemplaza al primero.
Acceso O(1)	Buscar por código es instantáneo — no recorre la colección, calcula la posición directamente.
Sin orden	Los materiales no salen en el orden en que se agregaron. El orden depende del hashCode.
get() → null	Si el código no existe, retorna null. No lanza excepción. Siempre verifica antes de usar el resultado.

■ ¿Por qué HashMap y no ArrayList para el catálogo?

Porque la pregunta de negocio es: **'dame el material con código X'**. Con ArrayList tendrías que recorrer toda la lista hasta encontrarlo — $O(n)$. Con HashMap vas directo al código — $O(1)$. El código del material es el identificador único natural: perfecto como clave.

■ El tipo declarado es Map. ¿Por qué no HashMap?

Por el mismo principio de la interfaz: programar contra el contrato, no la implementación. Si el día de mañana necesitas TreeMap (para tener el catálogo en orden alfabético), solo cambias la línea de instanciación — el resto del código no se toca.

En SistemaBiblioteca crea: `Map catalogo = new HashMap<>()`. Agrega un método `registrarMaterial(Material m)` que use `put()` con el código como clave. Agrega `buscarMaterial(String codigo)` que use `get()` y retorne el material o null.

■ ¿Qué pasa con el polimorfismo aquí?

El mapa es `Map<String, Material>` — pero los objetos reales son Libro y Revista. Cuando llamas `catalogo.values()` y recorres con `mostrarInfo()`, Java llama el `getTipo()` correcto de cada objeto en runtime. No hay un if que pregunte si es Libro o Revista — el objeto mismo sabe cómo responder.

Agrega un método `mostrarCatalogo()` que recorra el Map con `entrySet()` e imprima la info de cada material.

4

La lista de espera

ArrayList — Segunda colección

Cuando un material está prestado y alguien lo quiere, entra a una lista de espera. El primero en pedir debe ser el primero en recibir el material cuando esté disponible. Además, la misma persona puede estar en la lista de espera de varios materiales distintos — y un material puede tener varios usuarios esperando.

REGLAS CLAVE DEL ArrayList

Regla	Qué significa en la práctica
Orden garantizado	Los elementos salen en el mismo orden en que se agregaron. El primero en entrar es el primero en la lista.
Duplicados permitidos	La misma persona puede aparecer en múltiples listas de espera. Eso es correcto.
Tamaño dinámico	La lista crece con cada nuevo usuario que espera. No hay límite predefinido.
Acceso por índice	get(0) da el primero en la fila — el próximo en recibir el material.

■ ¿Por qué ArrayList y no HashSet para la lista de espera?

Porque el **orden importa**: quien llegó primero tiene prioridad. HashSet no garantiza ningún orden — las personas podrían recibir el material en orden arbitrario. Además, HashSet elimina duplicados: si la misma persona aparece dos veces (por error), un Set lo descartaría sin aviso. Aquí queremos saber exactamente quién está y en qué posición.

■ ¿La lista de espera es por material o es una sola lista global?

Por material. Cada material tiene su propia fila de espera. Esto sugiere que la estructura ideal es un **Map**: la clave es el código del material, el valor es la lista de usuarios en espera. Usa **HashMap** internamente.

En SistemaBiblioteca crea: `Map<String, ArrayList> listaEspera = new HashMap<>()`. **Agrega** `agregarEspera(String codigoMaterial, String usuario)`: si el material no tiene lista aún, créala. Si ya existe, agrega el usuario al final. **Agrega** `siguienteEnEspera(String codigoMaterial)`: retorna y elimina el primero de la lista.

■ ¿Por qué remove(0) en la lista de espera y no remove("nombreUsuario")?

Porque queremos al **primero en la fila**, no a alguien específico. `remove(0)` elimina por índice — el elemento en la posición 0, que es el más antiguo. `remove("nombre")` eliminaría la primera ocurrencia de ese nombre específico, lo que no respeta el orden de llegada.

■ **Trampa:** con `List`, `remove(0)` elimina por índice. Con `List`, `remove(3)` también elimina por índice (no el valor 3). Para eliminar el valor 3 en una lista de enteros: `remove(Integer.valueOf(3))`.

5

El registro de membresías

HashSet — Tercera colección

La biblioteca necesita saber rápidamente si un usuario tiene membresía activa antes de permitirle un préstamo. Un usuario no puede tener dos membresías — o tiene o no tiene. El orden en que se registraron no importa para este propósito.

REGLAS CLAVE DEL HashSet

Regla	Qué significa en la práctica
Sin duplicados	<code>add()</code> ignora silenciosamente el duplicado y retorna <code>false</code> . Un usuario no puede estar registrado dos veces.
Sin orden	El orden de iteración no está garantizado. No importa quién se registró primero.
<code>contains()</code> O(1)	Verificar si un usuario tiene membresía es instantáneo — no recorre toda la colección.
<code>add()</code> retorna boolean	<code>true</code> si se agregó (era nuevo), <code>false</code> si ya existía. Úsalo para detectar intentos de registro duplicado.

■ ¿Por qué HashSet y no ArrayList para las membresías?

Porque la pregunta de negocio es: '¿este usuario tiene membresía activa?' — sí o no. Para eso solo necesitas `contains()`, que en HashSet es O(1). Con ArrayList tendrías que recorrer toda la lista — O(n). Además, la unicidad es una regla de negocio: un usuario no puede tener membresía duplicada.

■ ¿Qué pasa si alguien intenta registrarse dos veces?

HashSet descarta el duplicado sin error ni excepción — silenciosamente. Por eso `add()` retorna boolean: si retorna `false`, ya estaba registrado. Puedes usar ese retorno para informar al usuario: 'ya tienes membresía activa'.

En SistemaBiblioteca crea: `Set miembros = new HashSet<>()`. **Agrega** `registrarMiembro(String usuario)`: usa `add()` y retorna el boolean — `true` si era nuevo, `false` si ya existía. **Agrega** `tieneMembresía(String usuario)`: usa `contains()`.

■ El tipo declarado es Set y no HashSet. ¿Por qué?

El mismo principio de siempre: programar contra la interfaz Set. Si después necesitas `LinkedHashSet` (para mantener el orden de registro) o `TreeSet` (para tener los miembros en orden alfabético), solo cambias la línea de instanciación. Todo lo que usa la referencia Set no necesita modificarse.

6

Integrar todo: SistemaBiblioteca + main

Las tres colecciones trabajando juntas

El sistema ya tiene sus piezas. Ahora las conectas en la clase principal y en un main que demuestre que todo funciona. El main no tiene lógica de negocio — solo crea objetos, los mete al sistema y llama los métodos. Toda la lógica vive donde debe.

■ ¿Qué debe hacer el main para probar el sistema completo?

Debe ejercitar cada colección y cada regla de negocio importante:

■ HashMap — catálogo

Crea al menos 2 libros y 2 revistas. Regístralos en el catálogo. Busca uno por código y muestra su info. Intenta buscar un código inexistente — verifica que no explota.

■ ArrayList — lista de espera

Marca un material como no disponible (prestado). Agrega 3 usuarios a su lista de espera en orden. Llama siguienteEnEspera() dos veces — verifica que salen en el orden correcto.

■ HashSet — membresías

Registra 3 usuarios como miembros. Intenta registrar el mismo usuario dos veces — verifica que registrarMiembro() retorna false. Verifica la membresía de un usuario registrado y de uno que no existe.

■ Polimorfismo

Llama mostrarCatalogo() — verifica que cada material muestra su tipo correcto (Libro o Revista) sin un if en el código de mostrarCatalogo().

■ ¿Qué acaba de pasar en mostrarCatalogo()?

Recorriste un Map donde los objetos son Libro y Revista. Llamaste mostrarInfo() sobre cada uno. mostrarInfo() llama getTipo() — que es abstracto en Material. En runtime, Java supo que este objeto es un Libro y llamó su getTipo(). Sin if, sin instanceof, sin cast. **Eso es vinculación dinámica. Eso es polimorfismo real.**

Criterio de decisión — para quedarse

La pregunta no es '¿cuál colección debo usar?'. La pregunta es '¿qué garantiza mi negocio?'. Cada requisito apunta a una estructura.

Si la pregunta de negocio es...	La colección es...	Porque...
¿Dame el material con código X?	HashMap	Búsqueda directa por clave en O(1)
¿Quién sigue en la fila de espera?	ArrayList	Orden garantizado, acceso por posición
¿Este usuario tiene membresía activa?	HashSet	Unicidad + contains() O(1)
¿Cuántos ejemplares hay y en qué orden llegaron?	ArrayList	Orden de inserción + duplicados válidos
¿Qué categorías están activas hoy (sin repetición)?	HashSet	Unicidad garantizada por contrato

Pregunta para la próxima clase

Si registras dos objetos Libro con el mismo título y el mismo código en un HashSet, ¿Java los considera duplicados y descarta el segundo?

Pruébalo. El resultado te va a sorprender — y la explicación requiere entender equals() y hashCode() en clases propias.

el